Projeto: Partida Sequencial de 4 Motores

Contexto — por que partir motores em sequência?

Em uma instalação real, é raro um único motor partir sozinho: linhas de produção, esteiras, bancos de bombas e compressores costumam ter **vários motores que precisam entrar em operação juntos**. Ligar todos ao mesmo tempo, no entanto, é uma péssima ideia por pelo menos três motivos:

- **Corrente de partida.** Um motor de indução, no instante em que liga, chega a puxar de **5 a 8 vezes** a corrente nominal de regime (o chamado inrush current), só caindo depois de alguns ciclos. Se 4 motores partem juntos, essas correntes de pico **se somam** — o disjuntor geral e o cabeamento, dimensionados para a operação em regime, podem não aguentar a soma dos picos.
- **Queda de tensão na rede.** Um pico de corrente grande o bastante causa um afundamento de tensão momentâneo que pode afetar outros equipamentos ligados na mesma instalação — CLPs reiniciando sozinhos, contatores “tremendo”, iluminação piscando.
- **Ordem de processo.** Em muitos processos, a ordem de partida importa por razões que não são elétricas: um exaustor precisa ligar **antes** do queimador que ele ventila; a esteira do **fim** de uma linha precisa ligar antes das esteiras anteriores, senão o material se acumula e entope o ponto de transferência.

A solução mais simples para o problema elétrico é a **partida sequencial** (também chamada de staggered start): em vez de ligar tudo de uma vez, cada motor entra em operação com um **atraso programado** em relação ao anterior, distribuindo os picos de corrente ao longo do tempo em vez de somá-los.

> Isso é diferente de um **soft-starter** ou **inversor de frequência** (VFD), que suaviza a partida de **um único** motor (rampa de corrente dentro da própria partida dele). Os dois recursos resolvem problemas parecidos, mas em níveis diferentes — nada impede de usar os dois juntos: soft-starter em cada motor, **e** partida sequencial entre eles.

Objetivo

Programar, no OpenPLC (ESP32, real ou com motores simulados por LEDs), a **partida sequencial de 4 motores**: ao pressionar um botão **Start**, os motores ligam um depois do outro, respeitando um atraso programado entre cada partida. Um botão **Parar** desliga todos os motores imediatamente e cancela a sequência, esteja ela terminada ou não.

Hardware sugerido

Papel	Componente	Pino	Endereço
Start da sequência	Botão pulsador (NA + pull-down 10kΩ)	GPIO23	`%IX0.0`
Parar tudo	Botão pulsador (NA + pull-down 10kΩ)	GPIO22	`%IX0.1`
Motor 1	LED + resistor (ou relé + motor)	GPIO2	`%QX0.0`
Motor 2	LED + resistor (ou relé + motor)	GPIO4	`%QX0.1`
Motor 3	LED + resistor (ou relé + motor)	GPIO16	`%QX0.2`
Motor 4	LED + resistor (ou relé + motor)	GPIO17	`%QX0.3`

> Se algum motor for acionado de verdade (não só um LED), lembre-se da regra de sempre isolar: **GPIO → módulo relé opto-isolado → contator → motor**, nunca GPIO acionando o contator direto (mesma regra da aula 04).

O que o sistema precisa fazer

1. **Sequência de partida.** Ao pressionar Start, o motor 1 liga imediatamente; o motor 2 liga um tempo programado depois; o motor 3, mais um tempo programado depois do motor 2; e assim por diante até o motor 4.
2. **Parada geral.** A qualquer momento — sequência em andamento ou já completa — pressionar Parar desliga **todos** os motores imediatamente.
3. **Reinício.** Depois de um Parar, um novo Start deve refazer a sequência **inteira** desde o motor 1 — não existe “continuar de onde parou”.
4. **Start repetido.** Pressionar Start de novo enquanto a sequência já está rodando não deve reiniciar nem bagunçar a sequência em andamento.
5. O tempo entre partidas deve ser **programável** (não precisa vir de um potenciômetro como no exercício do termostato — pode ser uma constante configurada no programa), mas o valor deve estar isolado num único lugar fácil de ajustar, não espalhado pela lógica.

Pergunta central: isso é uma Function ou um Function Block?

Antes de sair montando rungs, pare e decida: a lógica de sequenciamento (guardar em que ponto da sequência está, controlar os atrasos entre partidas) deveria ser escrita como uma **Function** ou como um **Function Block**, no sentido visto na aula 09?

Escreva, em uma ou duas frases, sua resposta e a justificativa **antes** de programar — depois compare com o que você efetivamente construiu.

Perguntas para te guiar (sem entregar a resposta)

1. Que informação essa lógica precisa **lembrar** de um ciclo de varredura para o outro? Uma Function, do jeito que foi definida na aula 09, consegue guardar esse tipo de informação?
2. Quantos comandos de saída (motor 1, 2, 3, 4) essa lógica precisa produzir **ao mesmo tempo**? Uma Function pode ter mais de uma saída de dado?
3. Você já viu, na aula 06, um bloco que "olha para trás" e sabe há quanto tempo uma entrada está em determinado estado. Ele pode ajudar aqui — mais de uma vez, com tempos diferentes?
4. Pensando no `meu_contador` da aula 09: ele usava, por dentro, um bloco `ADD` (uma Function) para acumular um valor num Function Block. Existe algo parecido que você possa fazer aqui — um Function Block "por dentro" do seu Function Block de sequenciamento?
5. O que precisa acontecer com o(s) temporizador(es) no exato momento em que alguém pressiona Parar, para garantir que um novo Start comece do zero e não "herde" nenhum tempo já contado?
6. Se você encapsular essa lógica num Function Block próprio (por exemplo, `PARTIDA_SEQUENCIAL`), caberia usá-lo **duas vezes** no mesmo programa, para controlar dois grupos de 4 motores de forma independente — do mesmo jeito que `MEU_CONTADOR0` e `MEU_CONTADOR1` na aula 09? O que isso te diz sobre a vantagem de ter escolhido Function Block?

Checklist de entrega

- [ ] Resposta escrita para a pergunta central (Function ou Function Block, com justificativa), feita **antes** de programar.
- [ ] Circuito montado (ou diagrama, se for só simulação com LEDs).
- [ ] Projeto no OpenPLC Editor com as variáveis, pinagem e lógica.
- [ ] Testado: sequência completa, com o atraso correto entre cada motor.
- [ ] Testado: Parar desliga tudo imediatamente, em qualquer ponto da sequência.
- [ ] Testado: depois de Parar, um novo Start refaz a sequência inteira desde o motor 1.
- [ ] Testado: Start repetido durante a sequência não a reinicia nem bagunça o andamento.
- [ ] Capturas de tela do modo debug mostrando a sequência progredindo (motores acendendo um a um, com os tempos/`ET` visíveis).